

面向 ARM64/鲲鹏处理器的 SDC 故障注入方案

基于 gem5 与 CHAOS 的分层、保护感知、可校准研究设计

研究判断	不做 ISA 天然可靠性排名；研究 ISA×微架构×保护×负载×故障模型交互
核心路线	CHAOS 基线修复 → ARM64 独有状态 → O3/LSQ → 鲲鹏 128B LLC → 实机 RAS 校准
证据截止	2026-08-25

目录

执行摘要	4
1. 研究范围、术语与证据边界	5
1.1 研究问题	5
1.2 核心术语	5
1.3 证据等级	5
1.4 研究限制	5
2. 学术前沿：从 AVF 到跨栈和机群校准	6
2.1 经典 AVF 阶段：识别 ACE 状态	6
2.2 微架构统计注入：真实传播替代保守上界	6
2.3 跨层关联：高层结果可能给出相反决策	6
2.4 从阵列到算术单元与门级缺陷	6
2.5 2026 年工具趋势：现代 gem5、模块化与安全注入	6
3. 产业前沿：从单次诊断到 factory-to-fleet	7
3.1 现场 SDC 的机制画像	7
3.2 产业趋势	7
4. 稳定规律与需要克制的趋势判断	8
4.1 稳定规律	8
4.2 趋势判断	8
5. x86-64 与 ARM64 的敏感性差异	9
5.1 可直接支持的跨 ISA 证据	9
5.2 机制层差异与假设	9
5.3 推荐的克制结论	9
6. 鲲鹏 920 的高价值靶点与优先级	10
7. CHAOS 与 gem5 基线审计	11
7.1 CHAOS 已有能力	11
7.2 必修问题	11
7.3 编译与运行矩阵	11
8. 建议框架：ARM-CHAOS-FI	12
8.1 总体架构	12
8.2 统一注入描述	12
8.3 O3 FaultInjectionPort	12

8.4 AArch64 扩展接口	12
8.5 鲲鹏 128B L3 建模	13
9. 故障模型	14
10. 实验工作负载与 oracle	15
10.1 工作负载组合	15
10.2 编译维度	15
10.3 Oracle	15
11. 结果分类与传播追踪	16
12. 统计设计与跨 ISA 归一化	17
12.1 抽样	17
12.2 配对跨 ISA 设计	17
12.3 从 AVF 到现场率	17
13. 分阶段 campaign	18
Phase 0: 版本冻结与工具审计	18
Phase 1: ARM64 架构级基线	18
Phase 2: ARM64 独有状态	18
Phase 3: O3 中枢与鲲鹏代理	18
Phase 4: x86-ARM64 配对对照	18
Phase 5: 鲲鹏实机校准	18
14. 关键微基准设计	19
15. 保护感知实验	20
16. 风险、反例与质量闸门	21
17. 预期成果与论文切分	22
论文 A: ARM-CHAOS 工具与方法	22
论文 B: ARM64 弱内存序与地址翻译的 SDC 图谱	22
论文 C: 鲲鹏 920 128B LLC 与 RAS 校准	22
18. 实施计划与资源	23
19. 最终建议	24
20. 主要来源与主张账本	25
21. 尚未解决的证据空白	26

执行摘要

本报告的直接结论是：公开证据不支持脱离具体微架构、保护机制和工作负载，笼统断言“ARM64 比 x86 更容易发生 SDC”或相反。最有可比性的跨 ISA 注入研究表明，在相同 gem5/GeFIN 方法与 MiBench 配置下，Arm 的物理寄存器堆、L1 指令缓存和 L1 数据缓存的瞬态 SDC AVF 通常高于 x86，且 L1D 差异最明显；但 Arm 的 Load/Store Queue 总 AVF 又低于 x86。更早的差分注入研究甚至发现，同一 ISA 在两个模拟器之间的差异可大于 x86 与 Arm 之间的差异。因此，研究对象应写成“ISA × 微架构 × 保护 × 工作负载 × 故障模型”的交互，而不是 ISA 标签的单变量排名。

产业工作的重心正在从随机软错误，转向制造测试逃逸、老化、边缘时序和未保护算术单元引起的可重复或条件触发 SDC；检测方式从单次离线诊断，演进为制造阶段筛查、周期性定向测试、低侵入在线测试、跨机群异常分析和应用端不变量联合。Meta/Google/Intel 等公开资料共同说明：单颗器件事件很稀少，但在百万级设备和长生命周期中成为必然运营问题；故障暴露强烈依赖指令、数据、温度、电压、频率和器件年龄。

对鲲鹏 920，公开资料能够确认的高价值差异靶点包括：TaiShan ARMv8.2-A 乱序核、64KB L1I/L1D、512KB 私有 L2、共享 L3、L3 采用 128B cache line（而常见 x86 为 64B）、多核/NUMA/片间互联，以及服务器级 ECC/RAS 能力。公开资料不足以确定内部 ROB、RAT、LSQ、预测器、ECC 覆盖边界和互连协议实现，因此这些靶点必须分成“公开事实”“合理代理模型”“待实机校准”三个证据等级。

推荐实施一条分层混合路线：

- 1 固定 gem5 与 CHAOS 版本，修复 CHAOS 的 ARM64 正确性问题，用架构寄存器、L1I/L1D 和内存建立可复现基线。
- 2 为 gem5 O3CPU 增加窄接口的字段级 FaultInjectionPort，优先扩展 AArch64 系统寄存器、TLB/页表遍历、LSQ 地址比较与转载转发、物理寄存器、NEON/FP、ROB/RAT。
- 3 为鲲鹏 920 的 64B L1/L2 与 128B L3 建立 sector/superline 代理模型；在完成精确适配前，不把全系统 128B 配置称为鲲鹏复现。
- 4 将 transient、intermittent、permanent、timing/delay 和 protection-aware fault 分开；将结果分为 Corrected/Contained、Masked、Latent、SDC、DUE/Crash、Hang。
- 5 以相同种子、动态窗口和位语义做 x86/ARM64 配对实验，同时报告条件传播概率与按 bit-time/entry-time 加权的风险；没有原始 FIT 或粒子截面时不换算现场故障率。
- 6 最终用鲲鹏实机的 RAS 错误记录、SEA/SError/CPER/EDAC/APEI 日志和定向硬件自检结果校准保护覆盖与错误分类。

本方案最适合形成三类论文贡献：一是对 CHAOS 的 ARM64 正确性审计与可复现扩展；二是 ARM64 特有状态及弱内存序路径的结构级 SDC 图谱；三是鲲鹏 920 的 128B LLC/NUMA/互连保护边界与 x86 64B LLC 的归一化比较。

1. 研究范围、术语与证据边界

1.1 研究问题

本报告回答四个问题：

- 1 截至 2026 年，SDC 故障注入的学术和产业前沿是什么？
- 2 哪些跨论文规律稳定，哪些趋势正在变化？
- 3 x86-64 与 AArch64 的 SDC 敏感性有哪些可证实差异，哪些只是待验证假设？
- 4 如何基于 gem5 与 [CHAOS](#) 设计一套面向 ARM64、特别是鲲鹏 920 的可执行注入方案？

1.2 核心术语

术语	本报告采用的含义
Fault	物理或模型层面的错误原因，如 SEU、stuck-at、时序违例、制造缺陷
Error	Fault 激活后形成的错误状态或错误值
Failure	Error 穿过系统边界形成错误服务结果
SDC	程序/系统产生错误结果，且未被既定硬件、OS 或应用检测链路发现
DUE	检测到但无法在线纠正或恢复的错误，包括异常、崩溃、panic 等
AVF	给定结构中的 fault 导致架构/程序可见错误的条件概率；不天然等于 SDC 概率
SDC AVF	给定注入已发生时，最终分类为 SDC 的比例
FIT	每十亿器件小时的故障数；需要 raw fault rate，不能只由 AVF 推出

1.3 证据等级

等级	定义	本方案中的用法
E1 直接证据	同条件跨 ISA 注入、厂商规范、源码或实机数据	可用于结论和必做靶点
E2 间接证据	单 ISA/单平台实验或架构机制证据	用于假设与优先级，不作总排名
E3 代理推断	gem5 参数近似、未公开微架构的合理映射	必须标注并做敏感性分析
E4 待校准	依赖鲲鹏内部 ECC/RAS/拓扑细节	必须由实机、RTL 或厂商资料验证

1.4 研究限制

公开资料中不存在同工艺、同节点、同保护开启状态、同负载和同统计口径的鲲鹏 920 与 Xeon/EPYC 现场 FIT 对照。Arm RAS 规范明确不要求系统中所有组件都成为 RAS node，也不规定系统必须达到何种可靠性水平；错误记录、纠正、延迟、不可恢复和不可包含能力多为实现定义。[Arm RAS System Architecture](#) 因此，本报告将“架构机制敏感性”与“具体产品现场故障率”严格分开。

2. 学术前沿：从 AVF 到跨栈和机群校准

2.1 经典 AVF 阶段：识别 ACE 状态

Mukherjee 等在 MICRO 2003 建立 AVF 与 ACE/un-ACE 状态分类：错误只有在影响 Architecturally Correct Execution 时才可能造成可见后果。错误路径、空闲状态、被覆盖值、NOP、预取和动态死代码等会产生屏蔽。该工作给出指令队列平均约 28%、执行单元约 9% 的 AVF，但它是 Itanium-like 结构、偏保守上界，并且 AVF 混合了 SDC 与 DUE。

随后，地址型结构的 AVF 被细化为 fill-read、read-read、read-evict 等生命周期区间。缓存数据、tag、DTB 和 store buffer 的敏感性不能只按容量或占用率估计；writeback、flush、scrub、替换和是否最终消费共同决定错误传播。

2.2 微架构统计注入：真实传播替代保守上界

GemFI、GeFIN、MaFIN、gem5-Approxilyzer、MeRLiN 和 gem5-MARVEL 推动了全系统微架构注入。它们的共同贡献是：在保留 OS、异常、预测执行、cache replacement 和应用算法屏蔽的条件下，观察 fault 从结构到输出的全过程。

MeRLiN 通过动态指令行为的等价 vulnerable interval 分组，把注入开销降低最高约三个数量级；它也展示了 ACE 法可能把物理整数寄存器堆风险严重高估。方法趋势由“均匀随机地打很多点”转向“按动态活跃性、位语义和传播等价类进行分层抽样”。

2.3 跨层关联：高层结果可能给出相反决策

《Demystifying the System Vulnerability Stack》表明，软件注入、架构级注入与微架构级注入可能给出不同甚至相反的保护收益排序。某些软件容错方案在 PVF/SVF 指标下看似降低风险，却在完整 cross-layer AVF 中使风险恶化。这说明保护评估必须保留 fault activation、硬件传播、架构提交、OS 处理和应用 oracle 的链路。

2.4 从阵列到算术单元与门级缺陷

2024-2025 年前沿明显转向未保护算术单元。Harpocrates 使用 hardware-in-the-loop 程序生成，针对现代 x86-64 算术单元生成短而高覆盖的检测程序；《From Gates to SDCs》把门级 stuck-at 模型嵌入 gem5，发现加法器错误大多导致 crash，而乘法器更可能被屏蔽或形成 SDC；Veritas 则把五种现代 x86 微架构的算术单元条件 SDC 概率与 Meta 机群缺陷率结合，强调不同 FU、指令类、负载和微架构之间差异显著。[Veritas / HPCA 2025](#)

该方向的重要变化是：缓存阵列通常可用 ECC/parity 低成本保护，而高频算术数据通路和组合逻辑难以在不影响性能的情况下完整保护。工业风险重点因此由“最大 SRAM 阵列”逐步转向“保护空洞 × 激活频率 × 可静默传播”的单元。

2.5 2026 年工具趋势：现代 gem5、模块化与安全注入

CHAOS 于 2026 年公开，强调面向现代 gem5 的模块化寄存器、缓存和内存注入；InjectV 等同期工作则把引导式注入用于 RISC-V 安全关键比较和控制流点。趋势是触发条件从随机 cycle 扩展到 PC、动态指令、事件、结构状态和安全语义。然而，CHAOS 论文实验只覆盖 RISC-V O3 + MiBench，其“所有 gem5 ISA 兼容”仍需 ARM64 编译和传播测试验证。[CHAOS 论文](#)

3. 产业前沿：从单次诊断到 factory-to-fleet

3.1 现场 SDC 的机制画像

Google、Meta 与 Intel 的公开资料一致指出：真正困难的 CPU SDC 往往不是完全随机、一次性的事件，而是制造缺陷、参数边缘、老化或局部单元缺陷在特定温度、电压、频率、指令序列和数据下被激活。Intel 的资料将这类错误定义为处理器产生错误结果而既有校验没有捕获，并将 DCDiag 用于制造和机群维护。[Intel Data Center Silent Data Errors](#)

Meta 公开的实践采用互补的三层检测：维护窗口中的 Fleetscanner、与生产负载共置的毫秒到秒级 Ripple，以及从异常分布反推异常核心的 Hardware Sentinel。其 2025 年总结强调，错误与工作负载、温度、频率、制造参数和寿命相关，必须贯穿器件生命周期反复测试。[Meta factory-to-fleet 实践](#)

3.2 产业趋势

趋势	研究含义
离线测试 + 在线低侵入测试 + 被动异常分析	gem5 方案不能只统计随机注入结果，还应输出可用于实机筛查的短签名测试
单位覆盖率转向生命周期覆盖	将温度、电压、频率、老化代理和测试周期纳入实验因素
厂商内部诊断转向跨厂接口	采用标准化 input/output/part-history 字段，便于与 OCP 指标对接
硬件 RAS 转向跨栈不变量	除异常日志外，引入 checksum、冗余计算、算法不变量和系统级数据校验
仅测“是否错”转向故障签名与根因定位	记录 target/field/bit/PC/动态指令/传播路径/症状/PMU/RAS 事件

[OCP Server Component Resilience](#) 正在标准化测试输入、输出、器件历史和效果指标。建议本项目从第一轮 campaign 就采用稳定 schema，而不是在结果出来后补字段。

4. 稳定规律与需要克制的趋势判断

4.1 稳定规律

- 1 **SDC 不等于总 AVF。** PRF 和 L1I 的非屏蔽错误常触发非法地址、非法指令或异常，因而以 crash/DUE 为主；L1D 或内存值损坏不一定扰动控制流，更容易静默到达输出。
- 2 **面积不是风险排名。** 条目是否有效、值是否活跃、生命周期、访问密度、覆盖/替换、保护机制和最终消费共同决定 SDC。
- 3 **瞬态和永久故障排序不同。** 单比特 transient、周期性 intermittent、stuck-at 和 gate-level defect 不能混在同一分母中。
- 4 **控制字段通常更容易产生快速失效，但少量合法域内错误可能高静默。** RAT 映射、LSQ 地址比较、byte-enable、权限位和 cache tag 是典型例子。
- 5 **单比特与多比特的结果分布不同。** 随机多比特打到寄存器或 L1 常迅速 crash；在未消费内存区则可能长期 masked/latent。真实 MBU 需具有空间相关性，不能随机散布。
- 6 **程序与编译器是一等因素。** 相同源程序在不同 ISA 上形成的动态指令数、寄存器压力、访存模式、替换行为和异常概率不同。
- 7 **注入点必须经过可达性、持久性和传播验证。** “写入 C++ 对象成功”不等于硬件语义上的 fault 已被消费。

4.2 趋势判断

过去	当前前沿
统一随机 cycle/bit	vulnerable interval、动态事件、语义位和定向触发
SRAM 阵列为中心	未保护算术、控制逻辑、互连和加速器数据通路
单层结果分类	hardware-to-software propagation 与 fleet 数据联合
transient 单比特为主	transient + MBU + intermittent + permanent + delay/timing
一次性离线 campaign	factory-to-fleet、生命周期与温压频条件化测试
给出单一 SDC 百分比	条件概率、置信区间、归一化风险与可校准 FIT 分离

5. x86-64 与 ARM64 的敏感性差异

5.1 可直接支持的跨 ISA 证据

gem5-MARVEL/ITC 2023 在同一模拟框架、15 个 MiBench 和相近配置下给出：

结构/故障	Arm SDC AVF 范围	x86 SDC AVF 范围	可用结论
PRF transient	0-6.9%	0-3.7%	该配置下 Arm 较高；总体仍以 crash 为主
L1I transient	0.3-9.9%	0.3-4.6%	该配置下 Arm 较高；SDC 远少于 crash
L1D transient	1.2-43%	0.7-32.6%	Arm 较高且 L1D 中 SDC 占主导
L1I permanent	0.1-2.3%	0.1-1.3%	x86 多数负载较低
L1D permanent	5.1-53.3%	4.4-64.7%	不存在稳定的 Arm>x86 总排序

来源：[Estimating the Failures and Silent Errors Rates of CPUs Across ISAs and Microarchitectures](#) 与 [gem5-MARVEL](#)。这些是**条件传播概率**，不含真实产品 raw FIT、工艺截面和 ECC 覆盖，不能换写成“鲲鹏现场故障率高于 Xeon”。

5.2 机制层差异与假设

差异维度	ARM64 假设	x86 假设	证据状态与实验要求
指令编码	A64 固定 32 位，随机翻转更可能仍形成合法指令或改变寄存器/立即数	x86 变长译码、prefix/uop 使控制影响复杂	不能仅凭编码断言；按“合法改写/异常/SDC”实测
寄存器语义	31 个 GPR、SP/XZR 编号语义、NZCV、EL 系统寄存器	较少架构 GPR、flags、分段/控制/MSR、部分写入语义	按角色和位域分层，禁止随机把保留位当有效位
内存序	Arm 较弱内存序、显式 DMB/DSB/ISB、exclusive monitor	x86 TSO 更强，lock/fence 语义不同	注入 LSQ 依赖、barrier 标记、exclusive 状态并使用 litmus oracle
地址翻译	TTBR/TCR/MAIR、ASID/VMID、stage-1/2、TLBI 广播	CR3/PCID、页表与 INVLPG 等	分系统寄存器、TLB entry、walker descriptor 三层注入
SIMD	鲲鹏 920 公开特性支持 ASIMD/NEON；128 位 lane/FP 位语义	SSE/AVX/AVX-512 依产品而变	相同算子和等价 lane 比较；不把 SVE 归于鲲鹏 920
RAS	Arm 给出统一 error record/SEA/SError 框架，覆盖实现定义	x86 MCA/poison/recovery 产业实现成熟但厂商/代际不同	比较“检测覆盖和归类”，不比较架构名义可靠性
LLC	鲲鹏 920 L3 128B line，L1/L2 64B	常见 x86 各级 64B	同时按 per-bit、per-entry、per-byte-hour 归一化
多核互连	鲲鹏集成多核、NUMA 与片间高速互联	UPI/Infinity Fabric 等产品差异大	对照应是拓扑/协议级，不是 ISA 级

5.3 推荐的克制结论

ARM64 相对 x86 的敏感性不是单一方向：在现有受控注入中，Arm 的 PRF/L1I/L1D transient SDC AVF 较高，而 LQ/SQ 较低；结构排序随 permanent fault、负载和实现改变。鲲鹏 920 的研究价值来自**ARM64 特有状态、128B LLC 故障域、多核/NUMA/互连路径和实现定义的 RAS 保护边界**，而不是预设“ARM64 更脆弱”。

6. 鲲鹏 920 的高价值靶点与优先级

公开服务器资料可确认鲲鹏 920 支持 ARMv8.1/v8.2、TaiShan 64 位乱序核、每核 64KB L1I + 64KB L1D + 512KB L2、共享 L3 和 ECC 能力；官方调优资料明确 L1/L2 为 64B line、L3 为 128B line。[鲲鹏 920 cacheline 资料](#)

优先级	靶点	关键字段/位	SDC 通道	证据等级
P0	AArch64 GPR/FP 架构寄存器	X0-X30、SP、NZCV、V0-V31 的 sign/exponent/mantissa/lane	值合法但错误、指针偏移、FP 有界偏差	E1/E2
P0	L1D 数据/tag/metadata	data、tag、valid/dirty、byte mask、ECC 前后	载入错误值、错误命中、脏数据写回	E1
P0	L1I 数据/tag	opcode、register field、immediate、tag	合法错误指令或控制流改变	E1
P1	LSQ/存储转发	地址、size、byte-enable、依赖、forward match	错源转发、漏依赖、内存序违规	E2，优先创新点
P1	TLB/页表 walker	VPN/PA、ASID/VMID、AP/XN/PXN、AttrIdx、valid	错页但合法访问、权限旁路、静默别名	E2，ARM64 特有语义
P1	系统寄存器	TTBR0/1_EL1、TCR_EL1、MAIR_EL1、SCTLR_EL1、VBAR_EL1	地址翻译/属性/异常入口被静默改变	E2/E3
P1	NEON/FP 数据通路	128 位 lane、FP exponent/mantissa、widen/narrow/saturate	多元素同时污染、数值仍有限	E2/E3
P1	L3 128B superline	两个 64B sector、tag/coherence/ECC syndrome	扩大数据域、错误共享/写回	E1 结构事实，E3 模型
P1	NUMA/片间互连	flit data、CRC、destination、transaction ID、poison	wrong-node data、丢/重复响应、错误路由	E2/E4
P2	RAT/PRF/ROB	map、free-list、ready、dest、done、fault、PC	合法错提交或旧值重用	E2/E3
P2	分支预测器	direction、BTB tag/target、RAS、history	多数性能/flush；极少错误提交需验证	E2
P2	barrier/exclusive monitor	DMB/DSB/ISB 标记、reservation granule、success bit	竞态下静默违反同步不变量	E2，ARM64 高价值
P3	中断/GIC/异常控制	pending/active/priority、vector select	漏中断、重复处理、错误恢复路径	E3/E4

不应把 SVE 作为鲲鹏 920 的既定单元。公开处理器 flags 与产品资料支持 ASIMD/NEON；SVE 应作为未来 ARMv9/其他鲲鹏代际的扩展研究，而不是本方案的基线事实。

7. CHAOS 与 gem5 基线审计

7.1 CHAOS 已有能力

CHAOS 由三个 SimObject 模块组成：CHAOSReg 修改 CPU architecture registers；CHAOSCache 针对 classic cache 数据路径；CHAOSMem 修改 AbstractMemory backing store。支持 bit flip、stuck-at-0、stuck-at-1，以及概率、时钟窗口、PC、mask 和 faulty-bits 等配置。论文报告的极低开销只来自 RISC-V O3 + MiBench，不能外推到 ARM full-system、Ruby/CHI 或大规模并发 campaign。

7.2 必修问题

问题	后果	修复与验收
未固定 gem5 tag/commit	默认分支漂移导致不可复现	固定 gem5 release commit 与 CHAOS commit，写入 manifest
CHAOSReg 使用 32 位 mask/bitset	AArch64 X 寄存器高 32 位和 128 位 NEON 无法覆盖	改为宽度感知 bit-vector；64/128 位单元测试
只选 IntRegClass/FloatRegClass	无 Misc/System/Vector/Predicate/CC	增加白名单 target class 与 ISA-aware field descriptor
架构寄存器接口	O3 推测态、PRF、RAT 不可见	明确 baseline 是 arch-state；另建 O3 FaultInjectionPort
PC target 逻辑需核验	可能在 PC 不匹配时仍允许注入	修复条件，测试命中/未命中/多线程/异常返回
stuck-at 的周期重写	可能不是物理永久故障语义，且与 squash/rename 不一致	定义 read-time/write-time/bit-cell 三种模型并分别验证
CHAOSCache 继承 classic Cache	不覆盖 Ruby CacheMemory/CHI	classic 与 Ruby 分支实现，不宣称通用兼容
packet hook 与驻留 block 混淆	只能改传输值，不能覆盖 tag/valid/coherence/EC C	增加 CacheBlk/CacheMemory 字段级接口
随机 cycle 概率注入	与真实 FIT、bit-time 暴露不等价	保存 conditional AVF；用 occupancy/bit-time 重新加权

7.3 编译与运行矩阵

ISA/模式	CPU	内存系统	目的
ARM SE	Atomic/Timing/Minor/O3	classic	最小可达性、速度与 CPU 模型兼容
ARM FS	Timing/O3	classic	Linux、异常、MMU、系统寄存器与 oracle
ARM FS	O3	Ruby MESI/CHI	一致性、TLB shutdown、互连与多核
x86 FS	O3	与 ARM 匹配	配对跨 ISA 对照
RISC-V SE	O3	classic	复现 CHAOS 论文 sanity baseline

每一格都必须验证：无注入黄金运行、强制注入发生、目标位读回、下一消费者可见、checkpoint restore、CPU switch、squash/rename/writeback/eviction 后语义，以及输出分类器。

8. 建议框架：ARM-CHAOS-FI

8.1 总体架构

建议不要继续为每个结构复制一套随机事件逻辑，而是把调度、目标选择、字段合法性、故障保持和结果记录分离：

层	组件	职责
Campaign	Campaign Controller	生成种子/配对样本、并发运行、超时、重试、manifest
Trigger	Trigger Engine	cycle、tick、PC、committed-inst、dynamic sequence、event/state 条件
Model	Fault Model Library	bitflip、MBU、stuck、intermittent、delay、read/write-path fault
Target	FaultInjectionPort	通过窄接口定位结构、entry、field、bit，不暴露容器/指针
Protection	Protection Model	pre-ECC、post-ECC、parity detect、poison、scrub/retry、unprotected
Trace	Propagation Recorder	activation→first read→commit→exception/RAS→output
Oracle	Outcome Classifier	Corrected、Contained、Masked、Latent、SDC、DUE、Hang

8.2 统一注入描述

每个 fault 用结构化记录表达：

```
fault_id, seed, isa, gem5_commit, chaos_commit, mode, cpu_model,
target, entry, field, bit_offset, original_value, injected_value,
fault_model, persistence, protection_stage, trigger_type, trigger_value,
pc, opcode, dyn_seq, thread, privilege_el, asid, vmid,
activation_tick, first_read_tick, commit_tick, ras_event,
outcome, oracle_delta, timeout_reason, workload_hash, config_hash
```

禁止随机修改 C++ 指针、迭代器、vector 长度或枚举到非法内存状态；这类行为模拟的是软件模拟器破坏，不是硬件 fault。控制字段应使用“合法域内错误值”，例如将 RAT map 改成另一个有效 PhysRegId、将 LSQ byte-enable 改成合法掩码、将 TLB PA 改成对齐页框。

8.3 O3 FaultInjectionPort

在 O3CPU 内部暴露窄而稳定的调试接口，由 CPU 自己完成合法访问：

```
injectPhysReg(phys_id, bitmask, phase)
injectRenameMap(arch_id, replacement_phys_id)
injectROB(seq, field, legal_value_or_mask)
injectLSQ(seq, field, legal_value_or_mask)
injectPredictor(component, entry, field, mask)
```

phase 至少区分 dispatch、issue、execute-complete、writeback、commit-before 和 commit-after。这样才能区分被 squash 的推测态 fault 与真正进入架构状态的 fault。

8.4 AArch64 扩展接口

系统寄存器模块使用白名单，不直接均匀随机遍历 MISCREG 枚举。重点寄存器包括 TTBR0/1_EL1、TCR_EL1、MAIR_EL1、SCTLR_EL1、VBAR_EL1、CONTEXTIDR_EL1、TPIDR* 与 NZCV。每个字段需标注 RW/RO、RES0/RES1、异常级、是否有写副作用和是否需要 TLBI/ISB。实验分为：

- 1 **NoEffect 状态损坏组**：只改变内部状态，用于纯 fault propagation。
- 2 **Architectural write 组**：触发正常写寄存器副作用，用于模拟控制状态被错误更新。

两组不能混合统计。

TLB 通过 `injectEntryFault(index/key, field, mask)` 修改 VPN、PA、ASID、VMID、权限、memory attributes、global 和 valid；页表 walker 单独修改 descriptor 或中间 walk state。必须分开 entry-resident fault、lookup-output transient 和 memory-resident page-table fault。

8.5 鲲鹏 128B L3 建模

gem5 当前把 cache-line size 作为 System 级参数，classic/Ruby 层级通常共享同一 block size；鲲鹏 920 却是 L1/L2 64B、L3 128B。因此采用三阶段：

- 1 **对照基线**：ARM 与 x86 都使用统一 64B line，研究 ISA/动态行为差异。
- 2 **边界代理**：保持 64B 系统 line，将相邻两个 L3 block 绑定为一个 128B fault domain；注入时同步目标、共享 superline id，并分别报告 sector 和 superline 结果。该模型不复现精确 replacement/coherence。
- 3 **精确扩展**：在 Ruby/CHI CacheMemory 中实现 sectorized 128B L3 superline，包含共同 tag/coherence 状态、两个 64B data sector、fill/evict/snoop 合并和 64B $\square$ 128B bridge；通过 litmus 与 directed cache test 验证。

在阶段 3 完成前，阶段 2 只能称为“鲲鹏 128B 故障域代理”，不能称为鲲鹏 L3 精确模型。

9. 故障模型

模型	参数	适用问题	主要限制
Transient SEU	单 bit、单时刻、单 entry	软错误条件传播、跨 ISA 可比	不等于真实粒子截面
Spatial MBU	同 word/line/lane 内 2/4/8 bits、距离分布	先进工艺多位 upset	禁止全结构独立随机散点
Intermittent	激活/休眠 burst、duty cycle、条件触发	温压频边缘、接触/老化	需记录每次激活，分母是 burst
Permanent stuck	cell/read-path/write-path stuck-at	制造缺陷/硬故障	周期重写不是唯一正确语义
Delay/timing	特定 opcode/data/温压频下超时或错误	临界路径和老化	gem5 非电路时序模拟，需门级/实机校准
Protection-aware	pre/post ECC、syndrome、parity、poison	RAS 覆盖与残余 SDC	保护参数多为实现定义

对 cache 至少拆为 data、tag、valid/dirty、coherence、replacement、ECC/parity syndrome；对执行单元拆为输入 latch、运算内部/门级代理、输出 latch；对寄存器拆为 storage cell、read output、write data。不同模型必须独立给出分母。

10. 实验工作负载与 oracle

10.1 工作负载组合

组	建议负载	目的
Directed microbench	GPR live-range、pointer、branch、LSQ forwarding、TLB、NEON lane、barrier/exclusive、cache superline	注入可达性、传播路径和单元签名
MiBench	与 CHAOS、GeFIN、gem5-MARVEL 对齐	复现和跨论文比较
SPEC CPU 2017	int/fp、rate/speed 子集	长程序与真实编译优化
PARSEC/SPLASH-3	多线程、共享内存与同步	弱内存序、coherence、NUMA
STREAM/Pointer chasing	带宽、cache/TLB/NUMA	L3、互连和内存路径
BLAS/CRC/crypto	NEON、整数/FP/CRC 数据通路	算术与 SIMD SDC 签名
Linux/OS stress	page migration、context switch、interrupt、fork/exec	EL1、TLB、异常与系统寄存器

10.2 编译维度

至少使用 GCC 与 LLVM 两条工具链，比较 **-O0/-O2/-O3**、是否自动向量化、是否 LTO；保留编译器版本、完整 flags、binary hash、动态指令数、代码大小和寄存器压力。x86/ARM64 的比较必须使用同一源代码、同一算法输入和尽可能等价的优化意图，但不能假设动态轨迹相同。

10.3 Oracle

- 1 确定性整数程序采用完整输出 hash、退出码和关键中间不变量。
- 2 浮点/AI 程序同时报告 bit-exact、ULP、相对误差、NaN/Inf 和任务级质量阈值；阈值内错误不能偷偷并入 masked。
- 3 并发程序用 linearizability、锁/计数器不变量、litmus forbidden outcome 和最终数据 hash。
- 4 OS 组记录 SEA/SError、kernel panic、signal、EDAC/APEI/CPER/RAS 日志和 watchdog。
- 5 每个 workload 至少做三次无 fault 黄金运行；若本身非确定，先建立合法输出分布，否则暂停 SDC 分类。

11. 结果分类与传播追踪

类别	判据
Corrected	硬件/模型纠正，输出与 golden 一致，并有 corrected 记录
Contained/Detected	错误被 parity/ECC/poison/RAS/OS 捕获，未形成错误输出
Masked	目标位已改变，但未被消费、被覆盖、被 squash 或算法屏蔽，输出正确
Latent	fault 在结束或观测窗口末仍存在且尚未消费；不并入 Masked
SDC	程序正常完成或服务继续，但输出/状态违反 oracle，检测链无告警
DUE/Crash	异常、signal、assert、panic、fatal、检测但不可恢复
Hang/Timeout	超过以 golden 指令数/时间分布定义的阈值；单独报告
Simulator error	gem5 assertion/非法宿主访问等工具故障；剔除并追根因

传播记录至少包含 activation、first read/use、进入提交、首次架构差异、首次 RAS/异常事件和最终 oracle。HVF 与 AVF 可在同一 seed/fault mask 上关联，但不能将“首次架构可见”直接算作 SDC。

12. 统计设计与跨 ISA 归一化

12.1 抽样

使用分层随机抽样，而不是对所有 bit 均匀抽样：

```
stratum = target × field-role × fault-model × protection-stage
× workload-phase × compiler × ISA/microarchitecture
```

第一轮每层可用 384 次作 95% 置信、最坏情形绝对误差约 $\pm 5\%$ 的筛选；关键层使用自适应样本量。对稀有 SDC，采用 Wilson 或 Jeffreys 区间，并以相对半宽、最小 SDC 事件数或预算作为停止条件。若 n 次中 0 个 SDC，95% 上界近似为 $3/n$ ，不能写成概率为零。CHAOS 论文中的 384/663/16587 仅对应二项比例的固定绝对误差目标，不应机械用于所有低概率层。

12.2 配对跨 ISA 设计

- 1 同一源代码和输入，保留不同二进制的动态统计。
- 2 同 seed 只表示“等价 fault role”，不强行用同物理 bit 编号。
- 3 触发点按 committed instruction percentile、ROI 内动态序列或语义事件对齐，不按绝对 tick 对齐。
- 4 同时报告：每次注入的条件 SDC 概率、每 10^9 committed instructions、每 bit-cycle/entry-cycle 和每被消费 fault 的传播概率。
- 5 使用分层 logistic/贝叶斯模型： $\text{outcome} \sim \text{ISA} + \text{target} + \text{workload} + \text{compiler} + \text{fault_model} + \text{interactions}$ ；将 seed/benchmark 作为随机效应或配对 block。
- 6 预先注册主要比较，控制多重检验；同时给出效应量与置信区间，不只给 p-value。

12.3 从 AVF 到现场率

```
estimated SDC FIT = raw fault FIT × occupancy/activation weight
× protection escape probability × conditional SDC AVF
```

如果没有鲲鹏特定 raw FIT、门级 defect distribution、ECC coverage 或粒子截面，只报告 conditional SDC AVF 和参数化区间。可用 sensitivity sweep 展示不同 raw FIT/coverage 下的排序是否改变，但不得给伪精确点估计。

13. 分阶段 campaign

Phase 0: 版本冻结与工具审计

目标: 形成可复现基线。**任务:** 固定 gem5/CHAOS commit; 建立容器/编译 manifest; 复现 RISC-V 示例; 完成 ARM SE/FS 编译矩阵; 修复 32 位 mask、PC trigger、stuck 语义和读追踪。**出口条件:** 每种 fault 均能在定向微基准中证明 original→injected→first-use→outcome 链路; 零 simulator error。

Phase 1: ARM64 架构级基线

目标: 用现成 CHAOS 范围得到可信的 ARM64 基准。**靶点:** GPR、FP/NEON、L1I、L1D、L2、memory。**模型:** single-bit transient、spatial MBU、read/write/cell stuck。**工作负载:** directed + MiBench。**出口条件:** 复现“L1D SDC 较高、L1I/PRF 多 crash”的方向性规律; 说明与文献差异。

Phase 2: ARM64 独有状态

目标: 形成可发表的 ARM64 结构敏感性图谱。**靶点:** TTBR/TCR/MAIR/SCTLR/VBAR、TLB、walker、ASID/VMID、barrier/exclusive、NEON lane。**工作负载:** page alias、permission、context switch、virtualization 可选、litmus、vector/FP。**出口条件:** 每一系统字段都有合法域和副作用模型; 能区分 SDC、权限/地址错误、SEA/SError 和工具错误。

Phase 3: O3 中枢与鲲鹏代理

目标: 研究 LSQ、RAT/PRF/ROB 和 128B L3 fault domain。**优先次序:** LSQ 地址/转发 → PRF/RAT → ROB 控制字段 → L3 sector/superline → coherence/互连。**出口条件:** 不通过破坏 C++ 容器制造结果; 同 fault 可在结构状态和 commit trace 中定位; 128B 模型明确代理误差。

Phase 4: x86-ARM64 配对对照

目标: 回答结构级差异而非 ISA 总排名。**设计:** 统一 CPU 宽度/队列/缓存容量和 64B line 的 controlled configuration; 另建产品近似 configuration; 配对 seeds/ROI/field role。**出口条件:** 给出主效应与交互效应、置信区间、模拟器/配置敏感性分析。

Phase 5: 鲲鹏实机校准

目标: 校准 protection escape 与日志链。**任务:** 枚举 RAS error records、EDAC/APEI/CPER、SEA/SError、BMC/IMU 日志; 使用厂商支持的错误注入/诊断接口; 运行由 gem5 生成的高覆盖短测试; 在安全边界内做温度/频率/负载扫描。**出口条件:** 明确哪些 gem5 fault 会被真实平台纠正、检测、poison 或静默; 更新参数区间。未经平台授权不得进行可能损害硬件或生产数据的电压/频率实验。

14. 关键微基准设计

编号	靶点	微基准核心	预期 oracle
BM-GPR	X 寄存器角色	live scalar、pointer、loop bound、return value 分组	精确 hash/异常分类
BM-NEON	NEON lane/FP 位	lane 独立累加、shuffle、widen/narrow、FMA	每 lane checksum + ULP
BM-LSQ	存储转发	同址/部分重叠/不同 size、store→load、alias	序列号和内存不变量
BM-WMM	弱内存序	MP/SB/LB/IRIW + DMB/DSB + LDXR/STXR	forbidden outcome
BM-TLB	TLB/Walker	ASID 切换、权限、页大小、AttrIndx、TLBI	VA→PA map 与 fault log
BM-SYS	系统寄存器	上下文切换前后保存/恢复，受控字段翻转	寄存器状态和异常入口
BM-L3-128	128B superline	两个 64B sector 分别读写/共享/写回	sector/superline checksum
BM-COH	coherence/NUMA	false sharing、producer-consumer、remote node	版本号/线性一致性
BM-FU	scalar/FP/crypto	opcode×operand pattern 定向算术	冗余计算和数学不变量

15. 保护感知实验

每个 cache/memory fault 至少形成四个对照：

- 1 **Raw/unprotected**：用于测结构传播上界。
- 2 **Pre-code injection**：fault 在 ECC/parity 生成前，通常随正确 code 一起存储；模拟错误源位于写入数据路径。
- 3 **Post-code injection**：fault 在数据与 code 存储后，模拟 cell upset；读取时触发纠正/检测。
- 4 **Post-check escape**：在检查之后修改消费值，用于刻画未保护 read/forward path，而不是伪装成 ECC 失效。

保护输出分别记录 corrected、detected/contained、poisoned、retry/scrub 和 escape。不能用“cache 有 ECC”把所有 cache fault 预先分类为非 SDC；也不能在无 ECC 模型时把 raw L1D AVF 解释成实际服务器残余 SDC。

16. 风险、反例与质量闸门

风险	典型错误结论	闸门
把 ISA 当产品	“Arm 比 x86 脆弱”	controlled config + product proxy 分开
CHAOS 假注入	bit 写入后立即被覆盖或目标未命中	original/readback/first-use 三重证据
32 位 mask	误称覆盖 AArch64	64/128 位单元测试
O3 架构态误当 PRF	误称研究 rename/投机	独立 FaultInjectionPort 与 phase
128B 全系统 line	误称复现鲲鹏 L3	sector/superline 代理标签或精确 bridge
随机控制字段	gem5 崩溃被计作 DUE	合法域约束，simulator error 单列
固定 timeout	慢负载误判 hang	按 golden 指令/时间分布设阈值
FP 容差混乱	小错误并入 masked	bit/ULP/任务级三套结果
0 次 SDC	声称概率为 0	报 95% 上界约 3/n
ECC 缺失	raw AVF 外推现场	protection stage 与参数化 FIT
多重比较	偶然显著	预注册、FDR/层次模型、效应量

17. 预期成果与论文切分

论文 A: ARM-CHAOS 工具与方法

贡献：CHAOS ARM64 审计；宽度感知寄存器、系统寄存器/TLB、保护感知注入；可复现数据 schema；与原 CHAOS RISC-V 结果对照。评价重点是正确性验证而非“新增更多靶点”。

论文 B: ARM64 弱内存序与地址翻译的 SDC 图谱

贡献：LSQ forwarding、barrier/exclusive、TLB/walker/system-register fault 的传播和检测边界；比较不同工作负载/编译器/异常级；给出能够用于实机筛查的短签名测试。

论文 C: 鲲鹏 920 128B LLC 与 RAS 校准

贡献：64B sector $\square$ 128B superline 模型；per-bit/per-entry/per-byte-hour 归一化；L3/coherence/NUMA 故障传播；用鲲鹏 RAS 日志和定向测试校准。若无法获得实机注错能力，应把题目降级为“Kunpeng-informed proxy model”，避免产品级结论。

18. 实施计划与资源

里程碑	建议周期	核心输出
M0 版本冻结与审计	2-3 周	manifest、编译矩阵、问题清单、directed tests
M1 ARM64 baseline	4-6 周	CHAOS 修复、P0 campaign、复现报告
M2 ARM64 独有状态	6-10 周	SysReg/TLB/Walker/NEON/WMM 模块
M3 O3/LSQ/128B L3	10-16 周	FaultInjectionPort、superline 代理/精确模型
M4 跨 ISA campaign	6-10 周	配对数据、层次统计、敏感性分析
M5 鲲鹏校准	4-8 周	RAS capability map、实机签名、参数更新

建议先用单核 SE/FS directed tests 完成功能正确性，再使用 checkpoint 和并行 worker 扩大样本。并发调度需保留每个 run 独立目录、确定 seed、超时原因和重跑标记；重跑不能覆盖原结果。

19. 最终建议

- 1 立即将当前课题表述从“寻找 ARM64 独有高敏感单元”改为“检验 ARM64/鲲鹏特有机制是否形成结构级 SDC 敏感性与保护空洞”。这能避免确认偏差。
- 2 第一创新靶点选 LSQ 地址比较/存载转发；它同时连接 Arm 弱内存序、alias 行为与静默错误值，是现有 CHAOS 未覆盖、文献相对稀疏且可在 gem5 O3 中精确验证的点。
- 3 第二创新靶点选 TLB/页表 walker/系统寄存器；它们具有 AArch64 明确字段语义，容易设计合法域内注入和强 oracle。
- 4 第三创新靶点选 NEON lane + 算术单元；与 Veritas/Harpocrates 的 x86 前沿形成直接对话，但必须区分寄存器存储 fault 与执行单元 defect。
- 5 鲲鹏 920 的 128B L3 是最扎实的产品差异点，但也是 gem5 建模难点。先做 paired-sector 代理，再决定是否投入精确 sectorized cache。
- 6 Cache/内存结果必须同时给 raw 与 protection-aware 两组；否则容易得到“L1D 最危险”却无法解释真实服务器 ECC/RAS 的结果。
- 7 最终产出应包含可复现代码、版本 manifest、fault schema、directed tests、campaign 数据和分析脚本；只给 SDC 百分比不足以支撑可信结论。

20. 主要来源与主张账本

主要主张	来源	日期/出版方	访问说明
AVF/ACE 经典定义与结构差异	A Systematic Methodology to Compute the AVFs for a High-Performance Microprocessor, Mukherjee et al.	MICRO 2003	用户提供 PDF; DOI 10.1109/MICRO.2003.1253181
地址结构生命周期 AVF	Computing AVFs for Address-Based Structures, Biswas et al.	ISCA 2005	用户提供 PDF
x86/Arm 差分注入与模拟器差异	Differential Fault Injection on Microarchitectural Simulators, Kaliorakis et al.	IISWC 2015	用户提供 PDF; DOI 10.1109/IISWC.2015.28
动态指令行为加速注入	MeRLiN, Kaliorakis et al.	ISCA 2017	用户提供 PDF; DOI 10.1145/3079856.3080225
跨栈方法可能给相反结论	Demystifying the System Vulnerability Stack, Papadimitriou & Gizopoulos	ISCA 2021	用户提供 PDF; DOI 10.1109/ISCA52012.2021.00075
跨 ISA PRF/L1I/L1D/LQ/SQ 与 SDC AVF	gem5-MARVEL	HPCA 2024	用户提供 PDF; 作者公开版本
现代 x86 定向硬件筛查	Harpocrates, Karystinos et al.	ISCA 2024	用户提供 PDF; DOI 10.1109/ISCA59077.2024.00045
门级算术 fault 到 SDC	From Gates to SDCs, Chatzopoulos et al.	DATE 2025	用户提供 PDF
现代 x86 算术单元 + fleet 数据	Veritas	HPCA 2025	用户提供 PDF; DOI 10.1109/HPCA61900.2025.00012
CHAOS 设计、模块与实验边界	CHAOS 论文、源码	2026	论文只实证 RISC-V
Arm RAS 不规定覆盖与可靠性水平	Arm RAS System Architecture JHI0100 A.c	Arm, 2026-04-06	官方非保密规范
鲲鹏 920 L3 128B line	Kunpeng Programming and Tuning Guide	华为鲲鹏社区	官方公开文档
鲲鹏 920 cache/ECC/互联公开规格	KunTai R722 白皮书	服务器厂商公开白皮书	产品级公开资料, 非微架构 TRM
x86 产业 SDC 定义与诊断	Intel Data Center Silent Data Errors	Intel, Rev.1.1, 2024-03	厂商技术论文
factory-to-fleet 检测趋势	Meta AI hardware reliability	Meta, 2025-07-22	产业实践; 加速器比例不外推 CPU
跨厂 SDC 测试接口标准化	OCP Server Component Resilience v1.0	OCP, 2024-09-27	公开规范
gem5 block size 为系统级参数	gem5 System 源码、历史设计说明	gem5	支持 64/128B, 但常规层级共享系统值

21. 尚未解决的证据空白

- 1 鲲鹏 920 内部 core pipeline、预测器、RAT/ROB/LSQ、cache ECC/parity 粒度和 RAS node 映射缺少公开 TRM。
- 2 没有鲲鹏 920 与同代 x86 的公开粒子束、门级 defect、同条件 field FIT 或 fleet denominator。
- 3 CHAOS 的 ARM64、full-system、Ruby/CHI 与多核兼容性尚无作者实验；本方案将其视为待验证软件假设。
- 4 gem5 的通用 ARM O3 参数不能复现 TaiShan v110 的未公开实现；所有产品结论必须参数化或实机校准。
- 5 温度/电压/频率引发的 delay fault 不能由普通 gem5 cycle model 物理生成；需要门级时序模型、硬件接口或经验触发函数。

研究停止标准是：上述报告章节已有足够的一手/用户提供论文支持；对最关键的跨 ISA 排名、Arm RAS 边界、CHAOS 能力和鲲鹏 128B L3 均完成了直接核查；继续广泛搜索不太可能改变方案主结构。未解决问题已明确转化为实验假设和校准任务，而非伪装成确定事实。